COURSE OUTLINE

Gennoor Tech
TRAIN. INNOVATE. BUILD.

Traffic and Flow Diagram Automation with Claude

From ping and traceroute evidence to an approved, versioned flow diagram

3 days (8 hours per day) | Intermediate | Instructor-led, hands-on labs | Capstone project

About this course

Who should attend

Network operations, NOC, load balancer, data centre and cloud network teams, and automation leads in enterprise IT in India and worldwide.

Prerequisites

- TCP/IP fundamentals and routing basics
- Hands-on familiarity with ping and traceroute
- Introductory Python and Git
- Exposure to at least one load balancer or ACI fabric is helpful, not required

What you will be able to do

- Explain what ping and traceroute can and cannot prove about a traffic path, and encode those limits in the diagram
- Set up Claude Code, the Claude Agent SDK and an MCP server that exposes read-only probe tools for this workflow
- Build deterministic Python probe runners that return typed path records Claude consumes through tool use and structured JSON outputs
- Enrich hops with F5, Citrix NetScaler, Radware Alteon, other load balancers (Anaya LB) and Cisco APIC context, and render evidence-linked Mermaid diagrams
- Publish diagrams to ServiceNow and GitHub behind scope limits, named approvals and an audit log, and detect path drift on a schedule

Tools and technologies

Claude Code and Agent SDK | Model Context Protocol (MCP) | Python

GennoorTech TRAIN. INNOVATE. BUILD. - https://gennoor.com for more info

Pydantic | Mermaid | F5 iControl REST and tmsh | Citrix NetScaler NITRO API | Radware Alteon / Cyber Controller | Cisco APIC REST API | ServiceNow Table API | GitHub pull requests | GitHub Actions

Table of contents

DAY 1 4 modules

MODULE 1

What a flow diagram must prove, and what probes can tell you

- Why hand-drawn traffic diagrams go stale: the request-to-diagram cycle in real NOC and change work
- What ping and traceroute actually show: ICMP and UDP probes, TTL expiry, RTT, per-hop loss, asymmetric return paths
- Where evidence goes dark: firewalls that drop ICMP, load balancers that hide pool members, ECMP and MPLS hops
- The teaching pattern for the course: Claude plans and explains, deterministic code probes, a person approves, everything is logged

HANDS-ON LAB
Trace the path from a lab client to a lab VIP by hand, record every hop and every gap, and write the brief for the pipeline you will build.

DELIVERABLE
Manual path record and pipeline brief.

GenNoor Tech TRAINING ACADEMY BUILD
For more info - https://gennoor.com

MODULE 2

Setting up Claude Code, the Agent SDK and MCP for this workflow

- Claude Code project layout for the diagram pipeline: CLAUDE.md, allowed tools, repository conventions
- Claude Agent SDK in Python: sessions, custom tool definitions, structured JSON outputs and permission hooks
- An MCP server exposing only read-only probe tools (ping, traceroute, DNS lookup) with scope lists and logging
- Secrets and credentials outside the prompt: environment variables and a vault for device API tokens

HANDS-ON LAB
Stand up the MCP server, connect it to Claude Code and the Agent SDK, and prove Claude can call a probe tool but cannot run an arbitrary shell command.

DELIVERABLE
Working Claude workbench with a read-only probe MCP server.

MODULE 3

Deterministic probe runners in Python

- Wrapping ping and traceroute with subprocess: timeouts, retries, max hops, packet counts and exit-code handling
- Parsers for Linux and Windows output into a typed path record, with unit tests against saved captures
- Handling the ugly cases: stars, duplicate hops, rate-limited routers and partially answering paths
- Claude requests probes through tool use; code runs them and returns structured results

HANDS-ON LAB
Implement and test the probe runners with Claude Code, inspect every diff, and run them against three lab targets with different failure modes.

DELIVERABLE
Tested probe runner library.

Gennoor Tech TRAINING · INNOVATE · BUILD
For more info - https://gennoor.com

MODULE 4

Path evidence model and structured outputs

- Pydantic schema for a path: source, target, hops, RTTs, loss, timestamps, probe ID and tool version
- Marking unknown segments explicitly rather than filling gaps; confidence per hop
- Prompting Claude to summarize a path record with mandatory citations to probe IDs
- Evidence bundle layout: raw output, parsed record, Claude summary, all versioned together

HANDS-ON LAB
Define the schema, feed Claude a path with missing hops, and verify the summary cites probe IDs and refuses to invent the gap.

DELIVERABLE
Path evidence schema and summarization agent.

DAY 2 4 modules

MODULE 5

Enriching hops with load balancer context

- F5: virtual servers, pools and pool members through iControl REST and tmsh list ltm virtual
- Citrix NetScaler lbvserver, servicegroup and binding objects through the NITRO API
- Radware Alteon virtual services via Cyber Controller and Alteon REST; a common adapter for other load balancers (Anaya LB)
- Matching a hop or VIP address to a device object so the diagram can show what sits behind it

HANDS-ON LAB
Build read-only adapters for the lab F5 and NetScaler, let Claude map a VIP to its pool members through tool use, and log every API call.

DELIVERABLE
Load balancer enrichment adapters.

Gennoor Tech TRAIN. INNOVATE. BUILD. For more infor https://gennoor.com

MODULE 6

Data centre fabric and cloud path context

- Cisco APIC REST API: tenants, bridge domains, EPGs and L3Outs that a path crosses
- Reading ACI endpoint and interface data to place a hop on a leaf or border leaf
- Hybrid paths: on-premises to cloud VPC or VNet, where traceroute stops and cloud route tables begin
- Deciding what to enrich and what to leave as a labelled unknown

HANDS-ON LAB

Query the lab APIC for the EPGs and L3Out a path traverses and attach fabric context to the evidence bundle.

DELIVERABLE

Fabric-aware path record.

MODULE 7

Generating the traffic and flow diagram with Mermaid

- Mermaid flowchart and sequence syntax for hop-by-hop traffic and request-response flow
- Claude drafts the diagram from the evidence bundle; a deterministic renderer validates the syntax
- Every node carries the probe ID that placed it; unknown segments are styled as unknown
- Diagram conventions: VIP, pool, firewall, fabric and cloud boundary shapes and a legend

HANDS-ON LAB

Generate Mermaid for the lab path, render it, break the evidence on purpose and watch the renderer reject an unsupported node.

DELIVERABLE

Evidence-linked Mermaid diagram generator.

GenNoor Tech
INNOVATE. BUILD.
For more info - https://gennoor.com

MODULE 8

Guardrails, approvals and audit for probing and publishing

- Scope lists: which prefixes and hostnames may be probed, enforced in code not in the prompt
- Rate limits and probe budgets so automated tracing never becomes a ping storm
- Named approver required before a diagram is written to the CMDB or merged into a shared repository
- Audit log of every tool call, device API query, model output and approval decision

HANDS-ON LAB
Add scope enforcement, rate limits and an approval gate, then attempt an out-of-scope probe and an unapproved publish and show both are refused and logged.

DELIVERABLE
Guardrail and audit layer.

DAY 3 3 modules

MODULE 9

Publishing to ServiceNow and GitHub

- ServiceNow Table API to update the incident, change or CI record that requested the diagram, and the Attachment API to attach the diagram and evidence bundle to it
- Flow Designer trigger: a request record starts the pipeline and receives the result
- GitHub pull request with the Mermaid file, evidence bundle and Claude-written summary; a person reviews and merges
- Repository layout for diagrams per application and per site

HANDS-ON LAB
Run the pipeline from a ServiceNow request to a GitHub pull request and complete the approval end to end.

DELIVERABLE
Request-to-publication workflow.

Gennoor Tech TRAIN INNOVATE BUILD For more info: https://gennoor.com

MODULE 10

Drift detection and scheduled re-mapping

- GitHub Actions schedule that re-runs probes within scope and compares against the published path
- Deterministic diff of path records; Claude explains the change in plain language
- Opening a pull request only when a path changed, with the diff and evidence attached
- Operator runbook: false drift from ECMP, maintenance windows and probe failures

HANDS-ON LAB
Schedule the re-mapping job, change the lab path, and review the drift pull request Claude explains.

DELIVERABLE
Drift detection job and runbook.

MODULE 11

Capstone: request to approved flow diagram, end to end

- A ServiceNow request in, probes run, hops enriched across F5, NetScaler, Radware, other load balancers (Anaya LB) and ACI, Mermaid diagram out
- No publication without a named approver; audit trail reviewed
- Acceptance criteria, limitations statement and team demo

HANDS-ON LAB
Capstone build and demo.

DELIVERABLE
End-to-end traffic and flow diagram pipeline.

Capstone

Capstone project

A repeatable pipeline that takes a flow diagram request from ServiceNow, runs ping and traceroute through deterministic code, enriches every hop from load balancer and ACI APIs, has Claude draft an evidence-linked Mermaid diagram, and publishes it to ServiceNow and GitHub only after a named approval, with drift detection on a schedule.

Assessment

Assessment area	Weight	Evidence
Working functionality	25%	Demonstrable end-to-end workflow in a repeatable lab

Gennoor Tech - https://gennoor.com

Security and safety controls	**20%**	Threat model, authorization boundaries, approval gates, safe failure
Testing and evaluation	**20%**	Automated tests, golden dataset, measured quality and known limitations
Engineering quality	**15%**	Clear interfaces, error handling, reproducible setup, maintainable code
Evidence and explainability	**10%**	Traceable findings, sources, assumptions, logs
Documentation and presentation	**10%**	Architecture diagram, README, runbook, demo, lessons learned

Engineering principles used throughout

- Model output is a proposal, not a fact: every finding and generated change is validated by deterministic checks and tests.
- Authorization lives outside the model: tool access is enforced by application code, scoped credentials and policy.
- Human approval for consequential actions: deploying a mitigation, changing production configuration, rotating credentials.
- Decisions, evidence, model and tool versions, approvals and outcomes are logged without recording secrets.
- A model adapter and a tool adapter layer keep every build provider-agnostic and enterprise-portable.
- Quality is measured: false positives, false negatives, reliability, latency, cost and safe failure, not just a demo.

© Gennoor Tech · Train. Innovate. Build.

Gennoor Tech TRAIN. INNOVATE. BUILD. For more info - https://gennoor.com